

Challenge Write-Up Template

Challenge Information

Student Name	Evallo, Daniella Kim N.
Section / Class	BSIT-3G
Challenge Name	bytemancy 1
Category	General Skills
Points / Difficulty	Easy
Date Solved	08/09/2026
Flag Format Given	PicoCTF{...}

1. Challenge Description

The challenge was similar to bytemancy 0, but this time ASCII decimal 101 had to be sent 1751 times. The characters had to be side-by-side with no spaces. The main problem was making sure I generated exactly the right number of characters before sending the input.

2. Initial Analysis / Reconnaissance

I first checked the challenge instructions and the hint. The important details were ASCII decimal 101 and the number 1751. Since decimal 101 is the lowercase letter 'e', the input needed to contain 1751 e characters in a row.

Typing 1751 characters manually would be easy to get wrong, so I followed the hint and used Python to generate the exact input.

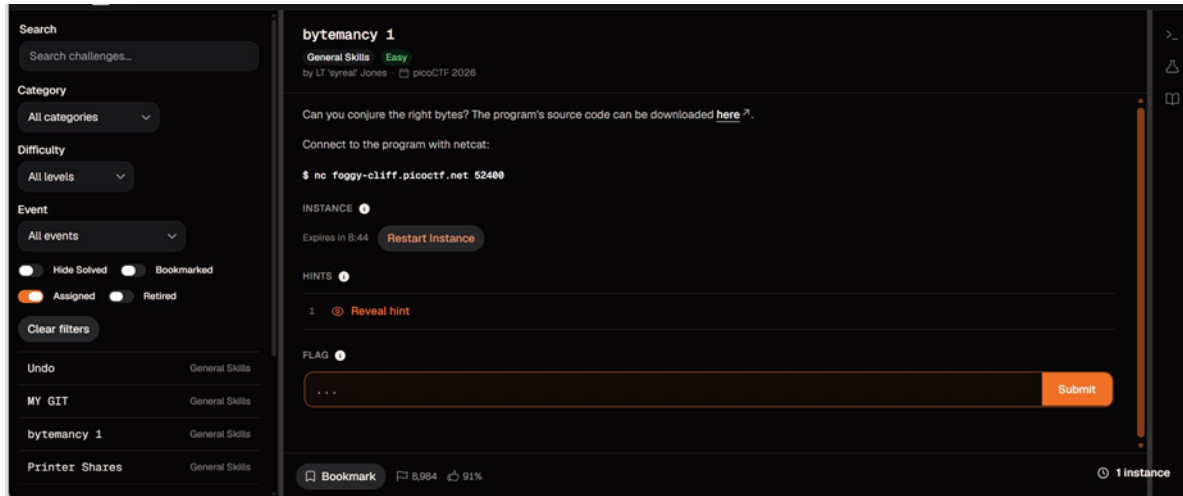
3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
Launch/ Restart Instance	Start or restart the bytemancy 1 challenge instance.	The challenge nstance became available.
python	Generate the required 1751 lowercase e characters.	The exact input was generated using <code>print('e' * 1751)</code> .
<code>print('e' * 1751)</code>	Create 1751 consecutive e characters without typing them manually.	The required repeated input was generated.
<code>nc foggy-cliff.picoctf.net 52400</code>	Connect to the remote bytemancy 1 challenge service.	The bytemancy 1 terminal opened.

4. Solution Walkthrough

Step 1

I read the challenge page and noted that **ASCII decimal 101** had to be sent **1751 times**, side-by-side with no spaces. The hint also said to use Python instead of copy-pasting.



Step 2

I used the same ASCII conversion from the first challenge. Decimal 101 represents the lowercase letter 'e'. That means the final input had to be 1751 consecutive e characters.



Step 3

Instead of typing the characters by hand, I used Python to generate them. The command used was:

print('e' * 1751). This makes the computer create exactly 1751 e characters, which reduces the chance of having too many or too few characters.

Step 4

I connected to the service using **netcat** and submitted the generated string. The service returned the flag, which matched the successful result shown in the challenge screenshot.



5. Flag

```
picoCTF{h0w_m4ny_e's??_706320e0}
```

6. Key Concept / What I Learned

This challenge showed me that a simple input requirement can become difficult when the same character has to be repeated many times. I learned that Python is useful for generating an exact number of characters and that netcat can be used to send the generated input to a remote challenge service.

7. References

[picoCTF 2026 | Denton's Blog](#)
